Replit Utvecklingsprompt: Kinab Core Concept - AI-Driven Backend och Planeringsplattform

---

1. Executive Summary

Vad är Kinab Core Concept?

Kinab Core Concept (Unicorn Platform) är en **AI-driven webbaserad plattform** för intelligent planering, schemaläggning och automation av fältservice-operationer. Plattformen är “hjärnan” som optimerar resurser, kommunicerar med kunder och ger beslutsunderlag till planerare.

Målgrupp

- **Primär:** Planerare och disponenter (50% av användartiden)
- **Sekundär:** Administratörer och chefer
- **Tertiär:** AI-systemet självt (autonom planering)

Huvudsyfte

1. **Autonom planering** - AI skapar optimala scheman utan manuell input
2. **Prediktiv analys** - Förutse underhållsbehov, tider och problem
3. **Intelligent kommunikation** - Automatisk kundkommunikation
4. **Kunskapshantering** - Centraliserad AI-driven kunskapsbas

Affärsmål

Mätetal	Nuvarande	Mål (12 mån)	Förbättring
Planeringstid/vecka	3-4 timmar	30 minuter	**87%**
Körsträcka	Baseline	-35%	**Bränsle**
Tidsestimering	±30%	±10%	**67%**
Kundsamtal	100/vecka	20/vecka	**80%**
Missade tidsfönster	15%	3%	**80%**

---

2. Nuvarande Funktionalitet (Behåll/Förbättra)

Stack och Arkitektur

```
╔═══════════════════════════════════════════════════════════════╗
║                          UNICORN PLATFORM                     ║
╚═══════════════════════════════════════════════════════════════╝
║                                                               ║
║  ┌─────────────┐  ┌─────────────┐  ┌─────────────┐  ┌─────────────┐  ║
║  ║  Frontend   ║  ║  Backend    ║  ║  AI Layer   ║  ║  External   ║  ║
║  ║  React/TS   ║  ║  Express.js ║  ║  OpenAI GPT ║  ║  Services   ║  ║
║  ║  Vite       ║  ║  PostgreSQL ║  ║  OR-Tools   ║  ║  Maps/SMS   ║  ║
║  ║  shadcn/ui  ║  ║  Drizzle ORM ║  ║  Weather API ║  ║  Fortnox   ║  ║
║  └─────────────┘  └─────────────┘  └─────────────┘  └─────────────┘  ║
║                                                               ║
║  ┌─────────────────────────────────────────────────────────┐  ║
║  ║                   MULTI-TENANT ISOLATION                ║  ║
║  ║  • Database-level isolation    • Dynamic tenant selection  ║  ║
║  ║  • API middleware validation   • Role-based access control ║  ║
║  ║  • White-label UI (CSS vars)   • Per-tenant configurations ║  ║
║  └─────────────────────────────────────────────────────────┘  ║
║                                                               ║
╚═══════════════════════════════════════════════════════════════╝
```

Befintliga Funktioner (✅ Behåll)

Planering och Schemaläggning

Funktion	Status	Beskrivning
WeekPlanner	✅ Fullständig	Drag-and-drop veckoplaner-ing
Auto-Fill Week	✅ Fungerande	Prioritetsbaserad + geo-grafisk klustering
Conversational AI Planner	⚠️ Grundläggande	GPT-4o-mini för naturligt språk
Travel Time Calculation	✅ Fungerande	Haversine-baserad restid-suppskattning
Weekly Goals	✅ Fullständig	Tid/Ekonomi/Antal-mål
Execution Codes	✅ Fullständig	Kompetens-till-uppgift-match-ning
Time Restrictions	✅ Fullständig	Parkeringsförbud, tömnings-dagar, etc.
Task Dependencies	✅ Fullständig	Beroendekedjor med negativ leveranstid

Orderhantering

Funktion	Status	Beskrivning
Orderkoncept	✅ Fullständig	Avrop, Schema, Abonnemang
Structural Tasks	✅ Fullständig	Komposit-uppgifter med del-steg
Article Hook System	✅ Fullständig	Automatiska artikelförslag
Metadata Writeback	✅ Fullständig	Automatisk metadata vid slutförande

Objekthantering

Funktion	Status	Beskrivning
Hierarkisk struktur	✅ Fullständig	Koncern → BRF → Fastighet → Rum → Kärl
EAV Metadata	✅ Fullständig	Flexibelt metadata-system
Object Images	✅ Fullständig	Bildhantering per objekt
Metadata History	✅ Fullständig	Ändringslogg med audit trail

Integration och Kommunikation

Funktion	Status	Beskrivning
SMS/Email	✅ Fungerande	Twilio + Resend
Fortnox	✅ Fungerande	Fakturaexport
Customer Portal	✅ Fullständig	Magic link, historik, chat
QR Issue Reporting	✅ Fullständig	Publik felanmälan

AI-Kapacitet (Nuvarande)

Funktion	Status	Beskrivning
AI Planning Suggestions	⚠️ Grundläggande	GPT-4o-mini för förslag
Weather-aware Scheduling	✅ Fungerande	7-dagars prognos
Anomaly Detection	⚠️ Grundläggande	Bakgrundsjobb
AICard Suggestions	⚠️ Grundläggande	Kontextbaserade kort

Directory-struktur

```
/
├── client/
│   ├── src/
│   │   ├── components/
│   │   │   ├── WeekPlanner/
│   │   │   ├── Dashboard/
│   │   │   ├── AICard/
│   │   │   └── ...
│   │   ├── pages/
│   │   │   ├── Planning.tsx
│   │   │   ├── Orders.tsx
│   │   │   ├── Objects.tsx
│   │   │   └── ...
│   │   ├── hooks/
│   │   ├── lib/
│   │   └── types/
├── server/
│   ├── routes/
│   ├── services/
│   ├── db/
│   │   ├── schema.ts
│   │   └── migrations/
│   └── index.ts
├── shared/
│   └── types/
└── package.json
```

---

3. AI-Drivna Funktioner - PRIORITERADE

KRITISK PRIORITET (Fas 1 - Månad 1-3)

Funktion 1: AI-Driven Automatisk Planering

Detaljerad Beskrivning

Fullständigt autonom planering som:
- Analyserar alla oplanerade ordrar
- Optimerar schema baserat på 20+ variabler
- Tar hänsyn till trafik, väder, kompetens
- Genererar optimal rutt per tekniker
- Balanserar arbetsbörda
- Förklarar sina beslut

Nuvarande vs. Framtid

```
NUVARANDE (Auto-Fill Week)                FRAMTID (AI-Driven)
┌──────────────────────────────┐          ┌──────────────────────────────┐
│ • Prioritetsbaserad sortering │          │ • ML-baserad tidsestimering   │
│ • Haversine geografisk kluster │          │ • OR-Tools optimering         │
│ • Kapacitetsmedveten          │          │ • Real-time trafikdata        │
│ • Overbooking-tolerans        │          │ • Väderprognos-integrering    │
│                               │          │ • Teknikerkompetens-matchning │
│ RESULTAT:                     │          │ • Historisk prestationsanalys │
│ Bra nog, men inte optimalt    │          │ • Förklarbar AI (XAI)         │
│                               │          │                               │
│                               │          │ RESULTAT:                     │
│                               │          │ 30-40% effektivare            │
└──────────────────────────────┘          └──────────────────────────────┘
```

Teknisk Implementation

AI Planner Service

```python
# server/services/ai-planner/planner.py (Pseudokod)

from ortools.constraint_solver import routing_enums_pb2
from ortools.constraint_solver import pywrapcp
import numpy as np
from sklearn.ensemble import RandomForestRegressor

class AIPlanner:
    def __init__(self):
        self.time_estimator = TimeEstimator()
        self.route_optimizer = RouteOptimizer()
        self.maps_client = GoogleMapsClient()
        self.weather_client = OpenMeteoClient()

    async def plan_week(
        self,
        orders: List[Order],
        technicians: List[Technician],
        constraints: PlanningConstraints
    ) -> PlanningResult:
        """
        Huvudfunktion för veckoplanering.

        Args:
            orders: Lista av oplanerade ordrar
            technicians: Lista av tillgängliga tekniker
            constraints: Begränsningar (tidsfönster, max timmar, etc.)

        Returns:
            PlanningResult med schema, rutter och metrics
        """

        # 1. Estimera tid per uppgift
        estimated_times = await self.time_estimator.predict_batch(
            orders=orders,
            technicians=technicians
        )

        # 2. Hämta externa data
        traffic_matrix = await self.maps_client.get_distance_matrix(
            origins=[t.home_location for t in technicians],
            destinations=[o.location for o in orders],
            departure_time=constraints.week_start
        )

        weather_forecast = await self.weather_client.get_forecast(
            location=constraints.region_center,
            days=7
        )

        # 3. Bygg optimeringsmodell
        model = self._build_optimization_model(
            orders=orders,
            technicians=technicians,
            estimated_times=estimated_times,
            traffic_matrix=traffic_matrix,
            weather_forecast=weather_forecast,
            constraints=constraints
        )

        # 4. Lös med OR-Tools
        solution = self.route_optimizer.solve(model)
```

```
        # 5. Generera rutter med Google Maps
        routes = await self._generate_routes(solution, technicians)

        # 6. Beräkna metrics och förklaringar
        metrics = self._calculate_metrics(solution, orders, technicians)
        explanations = self._generate_explanations(solution, orders)

        return PlanningResult(
            schedule=solution.schedule,
            routes=routes,
            metrics=metrics,
            explanations=explanations,
            alternatives=solution.alternatives[:3],
            confidence=solution.confidence
        )

    def _build_optimization_model(
        self,
        orders: List[Order],
        technicians: List[Technician],
        estimated_times: Dict[str, float],
        traffic_matrix: np.ndarray,
        weather_forecast: WeatherForecast,
        constraints: PlanningConstraints
    ) -> OptimizationModel:
        """
        Bygger OR-Tools-modell för Vehicle Routing Problem (VRP).
        """

        # Skapa data-modell
        data = {
            'num_vehicles': len(technicians),
            'num_locations': len(orders) + len(technicians),  # + depots
            'time_matrix': self._create_time_matrix(traffic_matrix, estimated_times),
            'time_windows': [(o.earliest, o.latest) for o in orders],
            'demands': [1 for _ in orders],  # Antal uppgifter
            'vehicle_capacities': [constraints.max_tasks_per_day for _ in techni-
cians],
            'execution_codes': [(o.required_codes, t.codes) for o in orders for t in
technicians],
            'weather_penalties': self._calculate_weather_penalties(orders, weath-
er_forecast),
        }

        # Skapa OR-Tools-modell
        manager = pywrapcp.RoutingIndexManager(
            data['num_locations'],
            data['num_vehicles'],
            data['depot_indices']
        )

        routing = pywrapcp.RoutingModel(manager)

        # Lägg till dimensioner
        self._add_time_dimension(routing, manager, data)
        self._add_capacity_dimension(routing, manager, data)
        self._add_execution_code_constraints(routing, manager, data)

        return OptimizationModel(
            manager=manager,
            routing=routing,
```

```
            data=data
        )
```

Express.js API Endpoint

```
// server/routes/ai-planning.ts

import { Router } from 'express';
import { AIPlanner } from '../services/ai-planner';

const router = Router();

router.post('/api/ai/plan-week', async (req, res) => {
  const { weekStart, technicianIds, constraints } = req.body;
  const tenantId = req.tenantId;

  try {
    // 1. Hämta ordrar och tekniker
    const [orders, technicians] = await Promise.all([
      orderService.getUnplannedOrders(tenantId, weekStart),
      technicianService.getByIds(tenantId, technicianIds),
    ]);

    // 2. Kör AI-planering
    const planner = new AIPlanner();
    const result = await planner.planWeek(orders, technicians, constraints);

    // 3. Logga för analys
    await planningLogService.log({
      tenantId,
      weekStart,
      ordersCount: orders.length,
      techniciansCount: technicians.length,
      planningTime: result.planningTime,
      metrics: result.metrics,
    });

    res.json({
      success: true,
      data: {
        schedule: result.schedule,
        routes: result.routes,
        metrics: result.metrics,
        explanations: result.explanations,
        alternatives: result.alternatives,
        confidence: result.confidence,
      },
    });
  } catch (error) {
    console.error('AI planning error:', error);
    res.status(500).json({
      success: false,
      error: 'Planning failed',
      details: error.message,
    });
  }
});

router.post('/api/ai/plan-preview', async (req, res) => {
  // Snabb förhandsvisning utan att spara
  // ...
});

router.post('/api/ai/plan-apply', async (req, res) => {
  // Tillämpa AI-genererat schema
  // ...
});
```

```
export default router;
```

React Frontend-komponent

```
// client/src/components/AIPlanningDialog.tsx

import { useState } from 'react';
import { useMutation } from '@tanstack/react-query';
import { Dialog, DialogContent, DialogHeader, DialogTitle } from '@/components/ui/dia-
log';
import { Button } from '@/components/ui/button';
import { Progress } from '@/components/ui/progress';
import { MapView } from './MapView';
import { SchedulePreview } from './SchedulePreview';

interface AIPlanningDialogProps {
  open: boolean;
  onClose: () => void;
  weekStart: Date;
  technicianIds: string[];
}

export function AIPlanningDialog({ open, onClose, weekStart, technicianIds }: AIPlan-
ningDialogProps) {
  const [result, setResult] = useState<PlanningResult | null>(null);
  const [step, setStep] = useState<'configure' | 'planning' | 'preview' | 'applied'>('
configure');

  const planMutation = useMutation({
    mutationFn: async (constraints: PlanningConstraints) => {
      const response = await fetch('/api/ai/plan-week', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ weekStart, technicianIds, constraints }),
      });
      return response.json();
    },
    onSuccess: (data) => {
      setResult(data.data);
      setStep('preview');
    },
  });

  const applyMutation = useMutation({
    mutationFn: async () => {
      const response = await fetch('/api/ai/plan-apply', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ schedule: result?.schedule, weekStart }),
      });
      return response.json();
    },
    onSuccess: () => {
      setStep('applied');
      setTimeout(onClose, 2000);
    },
  });

  return (
    <Dialog open={open} onOpenChange={onClose}>
      <DialogContent className="max-w-4xl max-h-[90vh] overflow-auto">
        <DialogHeader>
          <DialogTitle className="flex items-center gap-2">
            <span>🤖</span>
            <span>AI-Planering</span>
          </DialogTitle>
```

```
        </DialogHeader>

        {step === 'configure' && (
          <ConfigureStep
            onStart={(constraints) => {
              setStep('planning');
              planMutation.mutate(constraints);
            }}
          />
        )}

        {step === 'planning' && (
          <div className="flex flex-col items-center py-12">
            <div className="animate-pulse text-6xl mb-4">🤖</div>
            <h3 className="text-xl font-semibold mb-2">AI analyserar...</h3>
            <Progress value={planMutation.isPending ? 50 : 0} className="w-64" />
            <p className="text-muted-foreground mt-2">
              {planMutation.isPending && 'Optimerar rutter och schema...'}
            </p>
          </div>
        )}

        {step === 'preview' && result && (
          <div className="space-y-6">
            {/* Metrics */}
            <div className="grid grid-cols-4 gap-4">
              <MetricCard
                title="Effektivitet"
                value={`+${result.metrics.efficiencyGain}%`}
                color="green"
              />
              <MetricCard
                title="Körsträcka"
                value={`-${result.metrics.distanceReduction}%`}
                color="blue"
              />
              <MetricCard
                title="Balans"
                value={`${result.metrics.workloadBalance}%`}
                color="purple"
              />
              <MetricCard
                title="Konfidens"
                value={`${Math.round(result.confidence * 100)}%`}
                color="gray"
              />
            </div>

            {/* Map with routes */}
            <MapView routes={result.routes} />

            {/* Schedule per technician */}
            <SchedulePreview schedule={result.schedule} />

            {/* Explanations */}
            <div className="bg-muted p-4 rounded-lg">
              <h4 className="font-semibold mb-2">AI Förklaringar:</h4>
              <ul className="list-disc list-inside space-y-1">
                {result.explanations.map((exp, i) => (
                  <li key={i} className="text-sm">{exp}</li>
                ))}
              </ul>
            </div>
```

```
            {/* Actions */}
            <div className="flex justify-end gap-2">
              <Button variant="outline" onClick={() => setStep('configure')}>
                Justera
              </Button>
              <Button onClick={() => applyMutation.mutate()}>
                Tillämpa Schema
              </Button>
            </div>
          </div>
        )}

        {step === 'applied' && (
          <div className="flex flex-col items-center py-12">
            <div className="text-6xl mb-4">✅</div>
            <h3 className="text-xl font-semibold">Schema tillämpat!</h3>
            <p className="text-muted-foreground">Tekniker notifieras nu...</p>
          </div>
        )}
      </DialogContent>
    </Dialog>
  );
}
```

Användarflöden

Flöde 1.1: Automatisk veckoplanering

```
1. Planerare öppnar WeekPlanner
2. Klickar "🤖 AI-Planera Vecka"
3. Dialog öppnas:
   ╔═══════════════════════════════════════════════════════════════════╗
   ╟─ 🤖 AI-PLANERING                                                  ─╢
   ╟───────────────────────────────────────────────────────────────────╢
   ╟                                                                   ╢
   ╟ TEKNIKER ATT PLANERA:                                             ╢
   ╟ [✓] Johan        [✓] Anna        [✓] Erik        [ ] Maria (sjuk) ╢
   ╟                                                                   ╢
   ╟ BEGRÄNSNINGAR:                                                    ╢
   ╟ Max timmar/dag: [8] ▼                                             ╢
   ╟ Overbooking:    [110%] ▼                                          ╢
   ╟ Prioritera:     [Akuta först] ▼                                   ╢
   ╟                                                                   ╢
   ╟ ORDRAR ATT PLANERA: 127 st                                        ╢
   ╟ • 12 akuta                                                        ╢
   ╟ • 89 schema                                                       ╢
   ╟ • 26 avrop                                                        ╢
   ╟                                                                   ╢
   ╟ [         STARTA AI-PLANERING         ]                           ╢
   ╚═══════════════════════════════════════════════════════════════════╝

4. Planerare klickar "STARTA AI-PLANERING"
5. AI arbetar (30-60 sekunder):
   ╔═══════════════════════════════════════════════════════════════════╗
   ╟                                                                   ╢
   ╟                          🤖                                       ╢
   ╟                    AI analyserar...                               ╢
   ╟                                                                   ╢
   ╟ ▓▓▓
```

```
    │ • Johan fick flest pga hög effektivitet på denna objekttyp          │
    │ • Akuta ordrar placerade före lunch för alla tekniker               │
    │ • Trafik på E4 undviks måndag morgon                                │
    │                                                                     │
    │ [Justera manuellt] [Visa alternativ] [✓ TILLÄMPA SCHEMA]            │
    └─────────────────────────────────────────────────────────────────────┘

 7. Planerare klickar "TILLÄMPA SCHEMA"
 8. Schema sparas, tekniker notifieras
```

Acceptanskriterier

#	Kriterie	Mål	Test
1	Planeringstid	< 60 sekunder	Automatisk mätning
2	Effektivitetsvinst	> 20% vs manuell	A/B-test
3	Körsträckeminskning	> 25%	Jämför historik
4	Balans (tekniker)	85-115% av genoms-nitt	Beräkning
5	Respekterar con-straints	100%	Validering
6	Förklaringar	Minst 3 per planering	Kontroll
7	Alternativa planer	Minst 2	Kontroll

Funktion 2: Prediktiv Tidsestimering

Detaljerad Beskrivning

ML-modell som predikterar tid per uppgift baserat på:
- Tekniker (erfarenhet, hastighet, specialisering)
- Objekt (typ, komplexitet, historik)
- Kontext (väder, tid på dagen, trafik)
- Historisk data (faktiska tider från liknande uppgifter)

Teknisk Implementation

Machine Learning-modell

```python
# server/services/time-estimator/model.py

import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score
import joblib

class TimeEstimator:
    def __init__(self):
        self.model = RandomForestRegressor(
            n_estimators=100,
            max_depth=15,
            min_samples_split=5,
            random_state=42
        )
        self.scaler = StandardScaler()
        self.feature_names = []

    def train(self, historical_data: pd.DataFrame) -> dict:
        """
        Tränar modellen på historisk data.

        Args:
            historical_data: DataFrame med kolumner:
                - technician_id, object_type, object_count, weather,
                - temperature, time_of_day, day_of_week, actual_time

        Returns:
            dict med träningsmetrics
        """

        # Feature engineering
        X = self._extract_features(historical_data)
        y = historical_data['actual_time'].values

        # Spara feature-namn
        self.feature_names = X.columns.tolist()

        # Skala features
        X_scaled = self.scaler.fit_transform(X)

        # Träna modell
        self.model.fit(X_scaled, y)

        # Beräkna metrics
        cv_scores = cross_val_score(self.model, X_scaled, y, cv=5, scoring='neg_mean_a
bsolute_error')

        return {
            'mae': -cv_scores.mean(),
            'mae_std': cv_scores.std(),
            'feature_importance': dict(zip(self.feature_names, self.model.fea-
ture_importances_)),
            'training_samples': len(historical_data),
        }

    def predict(self, task: dict) -> dict:
        """
        Predikterar tid för en uppgift.
```

```
        Args:
            task: dict med uppgiftsinfo

        Returns:
            dict med estimat, konfidensintervall, förklaring
        """

        # Feature engineering
        features = self._extract_single_features(task)
        features_scaled = self.scaler.transform(features)

        # Prediktion från alla träd
        tree_predictions = [tree.predict(features_scaled)[0]
                            for tree in self.model.estimators_]

        # Beräkna estimat och intervall
        estimated_time = np.mean(tree_predictions)
        confidence_interval = np.percentile(tree_predictions, [10, 90])

        # Hitta liknande uppgifter
        similar_count = self._count_similar_tasks(task)

        return {
            'estimated_time': round(estimated_time, 1),
            'confidence_interval': [round(confidence_interval[0], 1),
                                    round(confidence_interval[1], 1)],
            'similar_tasks_count': similar_count,
            'explanation': self._generate_explanation(task, estimated_time),
        }

    def _extract_features(self, df: pd.DataFrame) -> pd.DataFrame:
        """Extraherar features från DataFrame."""

        features = pd.DataFrame()

        # Tekniker-features
        features['technician_experience_months'] = df['technician_experience_months']
        features['technician_avg_speed'] = df['technician_avg_speed']

        # Objekt-features
        features['object_type_encoded'] = df['object_type'].map(self.object_type_map)
        features['object_count'] = df['object_count']
        features['object_complexity'] = df['object_complexity']
        features['days_since_last_visit'] = df['days_since_last_visit']

        # Kontext-features
        features['temperature'] = df['temperature']
        features['precipitation'] = df['precipitation']
        features['hour_of_day'] = df['hour_of_day']
        features['day_of_week'] = df['day_of_week']
        features['is_morning'] = (df['hour_of_day'] < 12).astype(int)

        return features

    def _generate_explanation(self, task: dict, estimate: float) -> str:
        """Genererar human-readable förklaring."""

        base_time = self.base_times.get(task['object_type'], 30)
        diff = estimate - base_time

        explanations = []

        if diff > 5:
```

```
            if task['object_count'] > 3:
                explanations.append(f"Fler objekt ({task['object_count']} st) ökar
tiden")
            if task.get('temperature', 20) < 5:
                explanations.append("Kallt väder saktar ner arbetet")
            if task.get('technician_experience_months', 12) < 6:
                explanations.append("Ny tekniker, längre tid")
        elif diff < -5:
            if task.get('technician_avg_speed', 1.0) > 1.2:
                explanations.append(f"Snabb tekniker ({task['technician_name']})")

        if not explanations:
            explanations.append("Baserat på liknande uppgifter")

        return ". ".join(explanations)
```

API Endpoint

```
// server/routes/time-estimation.ts

router.post('/api/ai/estimate-time', async (req, res) => {
  const { task } = req.body;

  try {
    const estimator = new TimeEstimator();
    const result = await estimator.predict(task);

    res.json({
      success: true,
      data: result,
    });
  } catch (error) {
    res.status(500).json({ success: false, error: error.message });
  }
});

router.post('/api/ai/estimate-time/batch', async (req, res) => {
  const { tasks } = req.body;

  const estimator = new TimeEstimator();
  const results = await Promise.all(
    tasks.map(task => estimator.predict(task))
  );

  res.json({ success: true, data: results });
});

router.post('/api/ai/estimate-time/train', async (req, res) => {
  // Trigger omträning med nya data
  // Körs vanligtvis som scheduled job
});
```

Kontinuerlig Träning

```
// server/jobs/retrain-time-estimator.ts

import cron from 'node-cron';

// Kör omträning varje söndag kl 02:00
cron.schedule('0 2 * * 0', async () => {
  console.log('Starting weekly time estimator retraining...');

  // Hämta senaste 6 månadernas data
  const historicalData = await db.query(`
    SELECT
      t.technician_id,
      te.experience_months,
      te.avg_completion_speed,
      o.object_type,
      wo.object_count,
      o.complexity,
      EXTRACT(EPOCH FROM (wo.completed_at - wo.started_at))/60 as actual_time,
      w.temperature,
      w.precipitation,
      EXTRACT(HOUR FROM wo.started_at) as hour_of_day,
      EXTRACT(DOW FROM wo.started_at) as day_of_week
    FROM work_orders wo
    JOIN technicians te ON wo.technician_id = te.id
    JOIN objects o ON wo.object_id = o.id
    LEFT JOIN weather_log w ON DATE(wo.started_at) = w.date
    WHERE wo.completed_at IS NOT NULL
    AND wo.completed_at > NOW() - INTERVAL '6 months'
  `);

  // Träna modell
  const estimator = new TimeEstimator();
  const metrics = await estimator.train(historicalData);

  // Logga metrics
  await db.insert('model_training_logs', {
    model_name: 'time_estimator',
    trained_at: new Date(),
    metrics: JSON.stringify(metrics),
  });

  console.log('Retraining complete:', metrics);
});
```

Användarflöden

Flöde 2.1: Visa estimat i planering

```
1. Planerare drar order till tekniker i WeekPlanner
2. Estimat visas direkt:
   ┌──────────────────────────────────────────┐
   │ Tvätta 4 kärl, Palmgatan 17              │
   ├──────────────────────────────────────────┤
   │ ⏱ Estimat: 23 min (18-28 min)            │
   │                                          │
   │ ℹ Baserat på 47 liknande uppgifter av Johan │
   │   • Johan är 12% snabbare än genomsnittet │
   │   • Kallt väder: +3 min                  │
   └──────────────────────────────────────────┘
3. Estimat används för att beräkna dagstotal
```

Acceptanskriterier

#	Kriterie	Mål	Test
1	MAE (Mean Absolute Error)	< 5 minuter	Cross-validation
2	Förbättring vs standardtid	> 25%	A/B-test
3	API-svarstid	< 500ms	Lasttest
4	Batch-svarstid (100 tasks)	< 5s	Lasttest
5	Förklaringar	Genereras alltid	Kontroll

Funktion 3: Automatisk Kundkommunikation (Backend)

Detaljerad Beskrivning

Event-driven system som automatiskt kommunicerar med kunder:
- Bokningsbekräftelse
- ETA-uppdateringar
- Ankomst/avresa-meddelanden
- Rapport med bilder
- Tvåvägskommunikation (kund kan svara)
- AI-genererade personliga meddelanden

Teknisk Implementation

Event-Driven Arkitektur

```
// server/services/customer-communication/event-handler.ts

import { EventEmitter } from 'events';
import { TwilioClient } from './twilio';
import { ResendClient } from './resend';
import { OpenAI } from 'openai';

class CustomerCommunicationService extends EventEmitter {
  private twilio: TwilioClient;
  private resend: ResendClient;
  private openai: OpenAI;

  constructor() {
    super();
    this.twilio = new TwilioClient();
    this.resend = new ResendClient();
    this.openai = new OpenAI();

    // Registrera event handlers
    this.on('order.status_changed', this.handleStatusChange.bind(this));
    this.on('order.technician_on_route', this.handleOnRoute.bind(this));
    this.on('order.technician_arrived', this.handleArrived.bind(this));
    this.on('order.completed', this.handleCompleted.bind(this));
    this.on('deviation.created', this.handleDeviation.bind(this));
    this.on('sms.received', this.handleIncomingSMS.bind(this));
  }

  async handleOnRoute(event: OnRouteEvent) {
    const { order, technician, eta } = event;
    const customer = await this.getCustomer(order.customerId);

    if (!customer.preferences.enableSms) return;

    const message = await this.generateMessage('on_route', {
      technicianName: technician.name,
      eta: this.formatTime(eta),
      trackingUrl: this.generateTrackingUrl(order.id),
    });

    await this.sendSMS(customer.phone, message, order.id);
  }

  async handleArrived(event: ArrivedEvent) {
    const { order, technician } = event;
    const customer = await this.getCustomer(order.customerId);

    if (!customer.preferences.enableSms) return;

    const message = await this.generateMessage('arrived', {
      technicianName: technician.name,
    });

    await this.sendSMS(customer.phone, message, order.id);
  }

  async handleCompleted(event: CompletedEvent) {
    const { order, report, images } = event;
    const customer = await this.getCustomer(order.customerId);

    if (!customer.preferences.enableSms) return;

    // Generera AI-sammanfattning
```

```
    const summary = await this.generateCompletionSummary(order, report, images);

    const message = await this.generateMessage('completed', {
      summary,
      reportUrl: this.generateReportUrl(order.id),
    });

    await this.sendSMS(customer.phone, message, order.id);

    // Skicka även email med full rapport
    if (customer.preferences.enableEmail) {
      await this.sendCompletionEmail(customer.email, order, report, images);
    }
  }

  async handleDeviation(event: DeviationEvent) {
    const { order, deviation, estimatedCost } = event;
    const customer = await this.getCustomer(order.customerId);

    if (!customer.preferences.enableSms) return;

    const message = await this.generateMessage('deviation', {
      description: deviation.description,
      estimatedCost,
      imageUrl: deviation.imageUrl,
    });

    // SMS med möjlighet att svara JA/NEJ
    await this.sendSMS(customer.phone, message, order.id, {
      expectResponse: true,
      responseOptions: ['JA', 'NEJ'],
    });
  }

  async handleIncomingSMS(event: IncomingSMSEvent) {
    const { from, body, orderId } = event;

    // Tolka svar
    const response = body.trim().toUpperCase();

    if (['JA', 'YES', 'OK', 'J'].includes(response)) {
      await this.handlePositiveResponse(orderId, from);
    } else if (['NEJ', 'NO', 'N'].includes(response)) {
      await this.handleNegativeResponse(orderId, from);
    } else {
      // AI tolkar svaret
      const interpretation = await this.interpretResponse(body);
      if (interpretation.isPositive) {
        await this.handlePositiveResponse(orderId, from);
      } else if (interpretation.isNegative) {
        await this.handleNegativeResponse(orderId, from);
      } else {
        // Eskalera till manuell hantering
        await this.escalateToOperator(orderId, from, body);
      }
    }
  }

  private async generateMessage(type: string, data: any): Promise<string> {
    const templates: Record<string, (d: any) => string> = {
      on_route: (d) => `${d.technicianName} är på väg till er. Beräknad ankomst: ${d.e
ta}. Spåra: ${d.trackingUrl}`,
      arrived: (d) => `${d.technicianName} har anlänt!`,
```

```
      completed: (d) => `${d.summary} Se rapport: ${d.reportUrl}`,
      deviation: (d) => `Vi upptäckte: ${d.description}. Vill du att vi åtgärdar? (${d
.estimatedCost} kr) Svara JA/NEJ`,
      confirmation: (d) => `Tack! Vi bokar in ${d.action} ${d.date}.`,
    };

    return templates[type](data);
  }

  private async generateCompletionSummary(
    order: Order,
    report: Report,
    images: Image[]
  ): Promise<string> {
    const response = await this.openai.chat.completions.create({
      model: 'gpt-4o-mini',
      messages: [
        {
          role: 'system',
          content: `
            Generera ett kort, vänligt SMS-meddelande (max 100 tecken)
            som sammanfattar utfört arbete för en kund.
            Använd emojis sparsamt.
          `,
        },
        {
          role: 'user',
          content: JSON.stringify({
            description: order.description,
            completedItems: report.completedItems,
            imageCount: images.length,
          }),
        },
      ],
    });

    return response.choices[0].message.content!;
  }
}

export const communicationService = new CustomerCommunicationService();
```

Webhook för inkommande SMS (Twilio)

```
// server/routes/webhooks/twilio.ts

router.post('/webhooks/twilio/sms', async (req, res) => {
  const { From, Body, To } = req.body;

  // Validera Twilio-signatur
  if (!validateTwilioSignature(req)) {
    return res.status(401).send('Invalid signature');
  }

  // Hitta relaterad order
  const recentCommunication = await db.query(`
    SELECT order_id FROM customer_communications
    WHERE customer_phone = $1
    AND sent_at > NOW() - INTERVAL '24 hours'
    AND expects_response = true
    ORDER BY sent_at DESC
    LIMIT 1
  `, [From]);

  if (recentCommunication.length > 0) {
    communicationService.emit('sms.received', {
      from: From,
      body: Body,
      orderId: recentCommunication[0].order_id,
    });
  }

  // Twilio förväntar sig 200 OK
  res.status(200).send();
});
```

Användarflöden

Flöde 3.1: Komplett kundresa med automatisk kommunikation

```
TIDSLINJE: Order-livscykel med automatisk kommunikation

09:00 - Order skapas
        └─ [Ingen SMS - väntar på schemaläggning]

09:30 - Order schemaläggs till Johan, tisdag 10:00
        └─ SMS: "Ert besök är bokat tisdag 25/2 kl 10:00. Johan kommer."

TISDAG

09:45 - Johan trycker "Starta resa"
        └─ SMS: "Johan är på väg. Ankomst ca 10:15. Spåra: [länk]"

09:55 - ETA ändras pga trafik
        └─ SMS: "Johan försenad ca 10 min pga trafik. Ny ankomst: 10:25"

10:20 - Johan anländer (GPS detekterar)
        └─ SMS: "Johan har anlänt!"

10:45 - Johan slutför, tar bilder
        └─ SMS: "Klart! 4 kärl tvättade. Se bilder: [länk]"

10:46 - Johan rapporterar trasigt lysrör
        └─ SMS: "Vi upptäckte trasigt lysrör. Byta? (150 kr) Svara JA/NEJ"

10:50 - Kund svarar "JA tack gärna"
        ├─ [AI tolkar som positivt svar]
        ├─ SMS: "Tack! Byte av lysrör inbokat nästa vecka."
        └─ [System skapar automatiskt uppgift]
```

Acceptanskriterier

#	Kriterie	Mål	Test
1	SMS skickas inom	< 30 sekunder	Automatisk mätning
2	Korrekt ETA	±5 minuter	Jämför med ankomst
3	Tvåvägskommunikation	JA/NEJ hanteras	Testsamtal
4	AI-tolkning av svar	> 90% korrekt	Manuell validering
5	Minskning av kundsamtal	> 50%	Jämför historik

Funktion 4: Konversationell Planering

Detaljerad Beskrivning

Planerare kan prata/skriva med AI på naturligt språk:
- “Planera nästa vecka för Johan och Anna i Sigtuna”
- “Byt order 5 och 7”
- “Prioritera akuta idag”
- “Varför är Erik överbelastad?”

Teknisk Implementation

```
// server/services/conversational-planner/index.ts

import { OpenAI } from 'openai';
import { zodResponseFormat } from 'openai/helpers/zod';
import { z } from 'zod';

// Schema för strukturerade svar
const ConversationalActionSchema = z.object({
  intent: z.enum([
    'plan_week',
    'swap_orders',
    'prioritize',
    'explain',
    'show_schedule',
    'create_order',
    'unknown'
  ]),
  parameters: z.object({
    technicians: z.array(z.string()).optional(),
    orders: z.array(z.string()).optional(),
    area: z.string().optional(),
    dateRange: z.object({
      start: z.string(),
      end: z.string()
    }).optional(),
    priority: z.enum(['urgent', 'normal', 'low']).optional(),
    reason: z.string().optional(),
  }).optional(),
  clarificationNeeded: z.boolean(),
  clarificationQuestion: z.string().optional(),
  response: z.string(),
  actions: z.array(z.object({
    type: z.string(),
    data: z.any(),
  })).optional(),
});

class ConversationalPlanner {
  private openai: OpenAI;
  private conversationHistory: Map<string, Message[]> = new Map();

  constructor() {
    this.openai = new OpenAI();
  }

  async processMessage(
    conversationId: string,
    userMessage: string,
    context: PlanningContext
  ): Promise<ConversationalResponse> {
    // Hämta konversationshistorik
    const history = this.conversationHistory.get(conversationId) || [];

    // Bygg system-prompt med kontext
    const systemPrompt = this.buildSystemPrompt(context);

    // Skicka till GPT-4
    const response = await this.openai.chat.completions.create({
      model: 'gpt-4o',
      messages: [
        { role: 'system', content: systemPrompt },
        ...history,
```

```
        { role: 'user', content: userMessage },
      ],
      response_format: zodResponseFormat(ConversationalActionSchema, 'action'),
      tools: this.getTools(),
    });

    const result = JSON.parse(response.choices[0].message.content!);

    // Spara i historik
    history.push({ role: 'user', content: userMessage });
    history.push({ role: 'assistant', content: result.response });
    this.conversationHistory.set(conversationId, history);

    // Utför eventuella åtgärder
    if (result.actions && !result.clarificationNeeded) {
      for (const action of result.actions) {
        await this.executeAction(action, context);
      }
    }

    return result;
  }

  private buildSystemPrompt(context: PlanningContext): string {
    return `
      Du är en intelligent planeringsassistent för ${context.companyName}.

      NUVARANDE KONTEXT:
      - Datum: ${context.currentDate}
      - Aktiv vy: ${context.activeView} (vecka ${context.weekNumber})
      - Valda tekniker: ${context.selectedTechnicians.map(t => t.name).join(', ')}
      - Oplanerade ordrar: ${context.unplannedOrders.length} st
      - Akuta ordrar: ${context.urgentOrders.length} st

      TILLGÄNGLIGA TEKNIKER:
      ${context.allTechnicians.map(t => `- ${t.name}: ${t.area}, ${t.executionCodes.jo
in(', ')}`).join('\n')}

      REGLER:
      1. Svara alltid på svenska
      2. Var koncis men informativ
      3. Om du behöver mer info, fråga
      4. När du planerar, förklara varför
      5. Respektera constraints (tidsfönster, kompetens, etc.)

      ÅTGÄRDER DU KAN UTFÖRA:
      - plan_week: Planera en hel vecka
      - swap_orders: Byt ordrar mellan tekniker/dagar
      - prioritize: Ändra prioritet
      - create_order: Skapa ny order
      - explain: Förklara schema/beslut
      - show_schedule: Visa schema för tekniker/dag
    `;
  }

  private getTools() {
    return [
      {
        type: 'function',
        function: {
          name: 'ai_plan_week',
          description: 'Skapa AI-optimerad veckoplanering',
          parameters: {
```

```
            type: 'object',
            properties: {
              weekStart: { type: 'string' },
              technicianIds: { type: 'array', items: { type: 'string' } },
              constraints: { type: 'object' },
            },
          },
        },
      },
      {
        type: 'function',
        function: {
          name: 'swap_orders',
          description: 'Byt två ordrar',
          parameters: {
            type: 'object',
            properties: {
              order1Id: { type: 'string' },
              order2Id: { type: 'string' },
            },
          },
        },
      },
      {
        type: 'function',
        function: {
          name: 'get_schedule',
          description: 'Hämta schema för tekniker/dag',
          parameters: {
            type: 'object',
            properties: {
              technicianId: { type: 'string' },
              date: { type: 'string' },
            },
          },
        },
      },
    ];
  }
}
```

Frontend Chat-komponent

```
// client/src/components/ConversationalPlannerChat.tsx

import { useState, useRef, useEffect } from 'react';
import { useMutation } from '@tanstack/react-query';
import { motion, AnimatePresence } from 'framer-motion';

interface Message {
  id: string;
  role: 'user' | 'assistant';
  content: string;
  actions?: Action[];
  timestamp: Date;
}

export function ConversationalPlannerChat({ context }: { context: PlanningContext }) {
  const [messages, setMessages] = useState<Message[]>([]);
  const [input, setInput] = useState('');
  const messagesEndRef = useRef<HTMLDivElement>(null);
  const [conversationId] = useState(() => crypto.randomUUID());

  const sendMutation = useMutation({
    mutationFn: async (message: string) => {
      const response = await fetch('/api/ai/conversation', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          conversationId,
          message,
          context,
        }),
      });
      return response.json();
    },
    onSuccess: (data) => {
      setMessages(prev => [
        ...prev,
        {
          id: crypto.randomUUID(),
          role: 'assistant',
          content: data.response,
          actions: data.actions,
          timestamp: new Date(),
        },
      ]);
    },
  });

  const handleSend = () => {
    if (!input.trim()) return;

    setMessages(prev => [
      ...prev,
      {
        id: crypto.randomUUID(),
        role: 'user',
        content: input,
        timestamp: new Date(),
      },
    ]);

    sendMutation.mutate(input);
    setInput('');
```

```
  };

  useEffect(() => {
    messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' });
  }, [messages]);

  return (
    <div className="flex flex-col h-full border rounded-lg">
      {/* Header */}
      <div className="flex items-center gap-2 p-3 border-b bg-muted/50">
        <span className="text-2xl">🤖</span>
        <span className="font-semibold">AI Planeringsassistent</span>
      </div>

      {/* Messages */}
      <div className="flex-1 overflow-auto p-4 space-y-4">
        {/* Welcome message */}
        {messages.length === 0 && (
          <div className="text-center text-muted-foreground py-8">
            <p className="text-lg mb-4">Hej! Jag kan hjälpa dig planera.</p>
            <div className="flex flex-wrap justify-center gap-2">
              <SuggestionChip onClick={() => setInput('Planera nästa vecka')}>
                Planera nästa vecka
              </SuggestionChip>
              <SuggestionChip onClick={() => setInput('Visa akuta ordrar')}>
                Visa akuta ordrar
              </SuggestionChip>
              <SuggestionChip onClick={() => setInput('Vem är ledig imorgon?')}>
                Vem är ledig imorgon?
              </SuggestionChip>
            </div>
          </div>
        )}

        <AnimatePresence>
          {messages.map((msg) => (
            <motion.div
              key={msg.id}
              initial={{ opacity: 0, y: 20 }}
              animate={{ opacity: 1, y: 0 }}
              className={`flex ${msg.role === 'user' ? 'justify-end' : 'justify-
start'}`}
            >
              <div
                className={`max-w-[80%] rounded-lg p-3 ${
                  msg.role === 'user'
                    ? 'bg-primary text-primary-foreground'
                    : 'bg-muted'
                }`}
              >
                <p>{msg.content}</p>

                {/* Actions */}
                {msg.actions && msg.actions.length > 0 && (
                  <div className="mt-2 pt-2 border-t border-border/50">
                    {msg.actions.map((action, i) => (
                      <ActionButton key={i} action={action} />
                    ))}
                  </div>
                )}
              </div>
            </motion.div>
          ))}
```

```
        </AnimatePresence>

        {sendMutation.isPending && (
          <div className="flex justify-start">
            <div className="bg-muted rounded-lg p-3">
              <span className="animate-pulse">🤖 Tänker...</span>
            </div>
          </div>
        )}

        <div ref={messagesEndRef} />
      </div>

      {/* Input */}
      <div className="p-3 border-t">
        <div className="flex gap-2">
          <input
            type="text"
            value={input}
            onChange={(e) => setInput(e.target.value)}
            onKeyDown={(e) => e.key === 'Enter' && handleSend()}
            placeholder="Skriv ett meddelande..."
            className="flex-1 px-3 py-2 border rounded-lg focus:outline-none
focus:ring-2 focus:ring-primary"
          />
          <button
            onClick={handleSend}
            disabled={sendMutation.isPending}
            className="px-4 py-2 bg-primary text-primary-foreground rounded-lg
hover:bg-primary/90"
          >
            Skicka
          </button>
        </div>
      </div>
    </div>
  );
}
```

Användarflöden

Flöde 4.1: Konversationell planering

```
╔═══════════════════════════════════════════════════════════════════╗
║ 🤖 AI PLANERINGSASSISTENT                                         ╕
╞═══════════════════════════════════════════════════════════════════╡
│                                                                   │
│  ┌─────────────────────────────────────────────────────────────┐  │
│  │ Hej! Jag kan hjälpa dig planera.                            │  │
│  │                                                             │  │
│  │ [Planera nästa vecka] [Visa akuta] [Vem är ledig?]          │  │
│  └─────────────────────────────────────────────────────────────┘  │
│                                                                   │
│  👤 Planerare: "Planera nästa vecka för Johan och Anna i Sigtuna"  │
│                                                                   │
│  🤖 AI: "Jag hittar 47 ordrar i Sigtuna-området för vecka 9.       │
│         - 12 akuta                                                │
│         - 35 schema                                               │
│                                                                   │
│         Johan har 3 lediga dagar, Anna har 4.                     │
│         Ska jag inkludera alla ordrar?"                           │
│                                                                   │
│  👤 Planerare: "Ja, men prioritera akuta före lunch"               │
│                                                                   │
│  🤖 AI: [Analyserar i 30 sekunder]                                 │
│                                                                   │
│         "Klart! Här är min plan:                                  │
│                                                                   │
│         Johan: 23 uppgifter                                       │
│         - Mån-Tis: Akuta (12 st) - alla före 12:00                │
│         - Ons-Fre: Schema (11 st)                                 │
│         - Total körsträcka: 87 km                                 │
│                                                                   │
│         Anna: 24 uppgifter                                        │
│         - Mån: 6 uppgifter                                        │
│         - Tis-Fre: 18 uppgifter                                   │
│         - Total körsträcka: 92 km                                 │
│                                                                   │
│         [Visa på karta] [Godkänn] [Justera]"                      │
│                                                                   │
│  👤 Planerare: "Byt order 5 och 7 för Johan"                       │
│                                                                   │
│  🤖 AI: "Bytt! Order 5 (Storgatan 12) är nu på tisdag, order 7     │
│         (Palmgatan 3) på måndag. Ny restid: +5 min.               │
│         Vill du behålla ändringen?"                               │
│                                                                   │
│  👤 Planerare: "Ja, spara"                                         │
│                                                                   │
│  🤖 AI: "Schema sparat! Johan och Anna notifieras nu. ✅"          │
│                                                                   │
╘═══════════════════════════════════════════════════════════════════╛
```

Acceptanskriterier

#	Kriterie	Mål	Test
1	Förståelse av intent	> 90%	50 testfraser
2	Svarstid	< 3 sekunder	Automatisk mätning
3	Konversationsminne	Hela sessionen	Kontroll
4	Utförda åtgärder	100% korrekta	Validering
5	Svenska språkförståelse	Dialekt, slang hanteras	Manuell test

---

HÖG PRIORITET (Fas 2-3 - Månad 4-9)

Funktion 5: Prediktivt Underhåll

Detaljerad Beskrivning

ML-system som förutser när objekt behöver underhåll:
- Survival analysis för komponenter
- Time-series forecasting för slitage
- Proaktiva rekommendationer
- Kostnadsberäkning (proaktivt vs reaktivt)

Teknisk Implementation

```python
# server/services/predictive-maintenance/model.py

from lifelines import CoxPHFitter
from prophet import Prophet
import pandas as pd

class PredictiveMaintenanceService:
    def __init__(self):
        self.survival_model = CoxPHFitter()
        self.forecaster = Prophet()

    def predict_failure(self, object_id: str) -> dict:
        """
        Predikterar sannolikhet för fel inom 3 månader.
        """

        # Hämta objekthistorik
        history = self.get_object_history(object_id)

        # Feature engineering
        features = self.extract_features(history)

        # Survival analysis
        survival_prob = self.survival_model.predict_survival_function(features)

        # Sannolikhet för fel inom 90 dagar
        failure_prob_90d = 1 - survival_prob.loc[90].values[0]

        # Generera rekommendation
        if failure_prob_90d > 0.7:
            recommendation = "Byt nu (hög risk)"
            urgency = "high"
        elif failure_prob_90d > 0.4:
            recommendation = "Byt vid nästa besök (medel risk)"
            urgency = "medium"
        else:
            recommendation = "Bevaka (låg risk)"
            urgency = "low"

        # Kostnadsberäkning
        proactive_cost = self.calculate_proactive_cost(object_id)
        reactive_cost = self.calculate_reactive_cost(object_id)
        savings = reactive_cost - proactive_cost

        return {
            'object_id': object_id,
            'failure_probability_90d': failure_prob_90d,
            'recommendation': recommendation,
            'urgency': urgency,
            'proactive_cost': proactive_cost,
            'reactive_cost': reactive_cost,
            'potential_savings': savings,
            'component': 'gasdämpare',  # Identifierad komponent
            'next_check_date': self.calculate_next_check_date(failure_prob_90d),
        }
```

Användarflöden

Flöde 5.1: Prediktivt underhåll dashboard

```
╔═══════════════════════════════════════════════════════════════════════╗
║ 🔧 PREDIKTIVT UNDERHÅLL - DASHBOARD                                  ╒
╟───────────────────────────────────────────────────────────────────────╢
│                                                                       │
│ ⚠️ 12 objekt behöver åtgärd inom 3 månader                            │
│ 💰 Total potentiell besparing: 6,800 kr                               │
│                                                                       │
│ ┌─────────────────────────────────────────────────────────────────┐   │
│ │ # │ Objekt          │ Komponent     │ Risk │ Rekommendation │ Besparing │
│ ├───┼─────────────────┼───────────────┼──────┼────────────────┼──────────┤
│ │ 1 │ Palmgatan 17    │ Gasdämpare    │ 85%  │ Byt nu         │ 700 kr   │
│ │ 2 │ Storgatan 5     │ Dörrlås       │ 72%  │ Byt vid besök  │ 500 kr   │
│ │ 3 │ Björkvägen 12   │ Hjul          │ 65%  │ Byt vid besök  │ 400 kr   │
│ │ 4 │ Tallgatan 8     │ Lock          │ 58%  │ Bevaka         │ 350 kr   │
│ │...│ ...             │ ...           │ ...  │ ...            │ ...      │
│ └─────────────────────────────────────────────────────────────────┘   │
│                                                                       │
│ [Skapa underhållsorder för alla hög-risk] [Exportera CSV] [Konfigurera]
│                                                                       │
╚═══════════════════════════════════════════════════════════════════════╝
```

Funktion 6: Smart Kunskapsbas (Backend)

Detaljerad Beskrivning

Vektordatabas med AI-sökning för:
- Arbetsinstruktioner
- Objekthistorik
- Erfarna teknikers tips
- FAQ och manualer
- Automatisk inlärning från resolved issues

Teknisk Implementation

```
// server/services/knowledge-base/index.ts

import { Pinecone } from '@pinecone-database/pinecone';
import { OpenAI } from 'openai';

class KnowledgeBaseService {
  private pinecone: Pinecone;
  private openai: OpenAI;

  async ingestDocument(doc: Document): Promise<void> {
    // Chunka dokument
    const chunks = this.chunkDocument(doc.content, 500);

    // Generera embeddings
    const embeddings = await this.openai.embeddings.create({
      model: 'text-embedding-3-small',
      input: chunks,
    });

    // Spara i Pinecone
    const vectors = chunks.map((chunk, i) => ({
      id: `${doc.id}-${i}`,
      values: embeddings.data[i].embedding,
      metadata: {
        documentId: doc.id,
        tenantId: doc.tenantId,
        title: doc.title,
        chunk: chunk,
        chunkIndex: i,
      },
    }));

    await this.pinecone.index('knowledge').upsert(vectors);
  }

  async search(query: string, tenantId: string): Promise<SearchResult[]> {
    // Generera embedding för frågan
    const queryEmbedding = await this.openai.embeddings.create({
      model: 'text-embedding-3-small',
      input: query,
    });

    // Sök i Pinecone
    const results = await this.pinecone.index('knowledge').query({
      vector: queryEmbedding.data[0].embedding,
      topK: 10,
      filter: { tenantId },
      includeMetadata: true,
    });

    // Generera AI-svar
    const context = results.matches.map(m => m.metadata.chunk).join('\n\n');

    const answer = await this.openai.chat.completions.create({
      model: 'gpt-4o-mini',
      messages: [
        {
          role: 'system',
          content: `
            Du är en hjälpsam kunskapsassistent. Svara baserat på följande kontext:
            ${context}
```

```
            Om svaret inte finns i kontexten, säg det.
          `,
        },
        { role: 'user', content: query },
      ],
    });

    return {
      answer: answer.choices[0].message.content,
      sources: results.matches.slice(0, 3).map(m => ({
        title: m.metadata.title,
        snippet: m.metadata.chunk.substring(0, 200),
        score: m.score,
      })),
    };
  }
}
```

Funktion 7: Intelligent Lagerhantering

Detaljerad Beskrivning

AI-driven lagerhantering:
- Demand forecasting (ARIMA/Prophet)
- Lageroptimering per tekniker
- Automatiska beställningsförslag
- “Smart lastning” - AI föreslår vad tekniker ska ha med

Teknisk Implementation

```
# server/services/inventory/forecaster.py

from prophet import Prophet
import pandas as pd

class InventoryForecaster:
    def forecast_demand(self, article_id: str, days_ahead: int = 30) -> dict:
        """
        Forecasta efterfrågan för en artikel.
        """

        # Hämta historisk användning
        history = self.get_usage_history(article_id)

        # Förbered data för Prophet
        df = pd.DataFrame({
            'ds': history['date'],
            'y': history['quantity'],
        })

        # Träna modell
        model = Prophet(
            yearly_seasonality=True,
            weekly_seasonality=True,
        )
        model.fit(df)

        # Forecasta
        future = model.make_future_dataframe(periods=days_ahead)
        forecast = model.predict(future)

        return {
            'article_id': article_id,
            'forecast': forecast[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].tail(days
_ahead).to_dict('records'),
            'recommended_stock': self.calculate_recommended_stock(forecast),
            'reorder_point': self.calculate_reorder_point(forecast),
        }
```

Funktion 8: Immersiv Kundportal (Backend)

Detaljerad Beskrivning

Real-time kundupplevelse:
- Live-tracking av tekniker på karta
- WebSocket-baserade uppdateringar
- Bilddelning i realtid
- Tvåvägschat

Teknisk Implementation

```
// server/services/customer-portal/realtime.ts

import { WebSocket, WebSocketServer } from 'ws';

class CustomerRealtimeService {
  private wss: WebSocketServer;
  private connections: Map<string, WebSocket> = new Map();

  constructor() {
    this.wss = new WebSocketServer({ port: 8080 });

    this.wss.on('connection', (ws, req) => {
      const orderId = this.extractOrderId(req);
      this.connections.set(orderId, ws);

      ws.on('close', () => {
        this.connections.delete(orderId);
      });
    });
  }

  async broadcastTechnicianPosition(orderId: string, position: Position) {
    const ws = this.connections.get(orderId);
    if (ws && ws.readyState === WebSocket.OPEN) {
      ws.send(JSON.stringify({
        type: 'position_update',
        data: {
          lat: position.lat,
          lng: position.lng,
          eta: position.eta,
          timestamp: new Date(),
        },
      }));
    }
  }

  async broadcastStatusUpdate(orderId: string, status: string, details: any) {
    const ws = this.connections.get(orderId);
    if (ws && ws.readyState === WebSocket.OPEN) {
      ws.send(JSON.stringify({
        type: 'status_update',
        data: { status, details, timestamp: new Date() },
      }));
    }
  }

  async broadcastImage(orderId: string, imageUrl: string, caption: string) {
    const ws = this.connections.get(orderId);
    if (ws && ws.readyState === WebSocket.OPEN) {
      ws.send(JSON.stringify({
        type: 'image_shared',
        data: { imageUrl, caption, timestamp: new Date() },
      }));
    }
  }
}
```

MEDEL PRIORITET (Fas 4 - Månad 10-12)

Funktion 9: Intelligent Overbooking

Detaljerad Beskrivning

Statistisk analys för smart overbooking:
- Monte Carlo-simulering
- Risk-beräkning per tekniker
- Dynamiska rekommendationer

```
# server/services/overbooking/simulator.py

import numpy as np

class OverbookingSimulator:
    def calculate_overbooking_recommendation(
        self,
        technician_id: str,
        planned_hours: float
    ) -> dict:
        """
        Beräknar optimal overbooking-nivå.
        """

        # Hämta historisk prestanda
        history = self.get_technician_performance(technician_id)

        # Monte Carlo-simulering
        n_simulations = 1000
        completion_times = []

        for _ in range(n_simulations):
            # Simulera varje uppgift
            simulated_time = sum([
                np.random.normal(task['estimated'], task['std'])
                for task in history['tasks']
            ])
            completion_times.append(simulated_time)

        # Beräkna risk vid olika overbooking-nivåer
        risks = {}
        for overbooking in [100, 105, 110, 115, 120]:
            target_time = planned_hours * (overbooking / 100)
            risk = sum(1 for t in completion_times if t > target_time * 60) /
n_simulations
            risks[overbooking] = risk

        # Hitta optimal nivå (max 10% risk)
        optimal = max([k for k, v in risks.items() if v <= 0.1], default=100)

        return {
            'technician_id': technician_id,
            'recommended_overbooking': optimal,
            'risk_at_recommended': risks[optimal],
            'extra_minutes_per_day': planned_hours * 60 * (optimal - 100) / 100,
            'risk_profile': risks,
        }
```

Funktion 10: Advanced Analytics Dashboard

Detaljerad Beskrivning

Omfattande analytics med:
- Real-time KPI-tracking
- Prediktiv analys
- Drill-down-möjligheter
- Exporterbara rapporter

4. Teknisk Arkitektur

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                          KINAB CORE CONCEPT ARKITEKTUR                       │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│   ┌──────────────────────────────────────────────────────────────────────┐   │
│   │                    WEBB APP (React + TypeScript)                     │   │
│   ├──────────────────────────────────────────────────────────────────────┤   │
│   │  Pages              │  Components           │  Hooks                 │   │
│   │  • Planning         │  • WeekPlanner        │  • useAIPlanning       │   │
│   │  • Orders           │  • ConversationalChat │  • useTimeEstimation   │   │
│   │  • Analytics        │  • PredictiveUI       │  • useKnowledgeSearch  │   │
│   │  • Maintenance      │  • MapView            │  • useRealtime         │   │
│   └──────────────────────────────────────────────────────────────────────┘   │
│                                     │                                        │
│                                     ▼                                        │
│   ┌──────────────────────────────────────────────────────────────────────┐   │
│   │                        API GATEWAY (Express.js)                      │   │
│   │                                                                      │   │
│   │  /api/ai/plan-week          /api/ai/estimate-time                    │   │
│   │  /api/ai/conversation       /api/ai/knowledge/search                 │   │
│   │  /api/ai/predict-maintenance /api/ai/inventory/forecast              │   │
│   └──────────────────────────────────────────────────────────────────────┘   │
│                                     │                                        │
│           ┌─────────────────────────┼─────────────────────────┐              │
│           ▼                         ▼                         ▼              │
│   ┌──────────────────┐      ┌──────────────────┐      ┌──────────────────┐   │
│   │    AI SERVICES   │      │  BUSINESS LOGIC  │      │    INTEGRATIONS  │   │
│   ├──────────────────┤      ├──────────────────┤      ├──────────────────┤   │
│   │ • AI Planner     │      │ • Order Service  │      │ • Google Maps    │   │
│   │   (OR-Tools)     │      │ • Schedule Engine │      │ • Open-Meteo     │   │
│   │ • Time Estimator │      │ • Metadata System │      │ • Twilio SMS     │   │
│   │   (RandomForest) │      │ • Invoice Engine  │      │ • Resend Email   │   │
│   │ • NLP Service    │      │ • Reporting      │      │ • Fortnox        │   │
│   │   (GPT-4)        │      │                  │      │ • OpenAI         │   │
│   │ • Vision Service │      │                  │      │ • Pinecone       │   │
│   │ • Maintenance    │      │                  │      │                  │   │
│   │   (CoxPH/Prophet) │      │                  │      │                  │   │
│   └──────────────────┘      └──────────────────┘      └──────────────────┘   │
│                                     │                                        │
│                                     ▼                                        │
│   ┌──────────────────────────────────────────────────────────────────────┐   │
│   │                            DATA LAYER                                │   │
│   │                                                                      │   │
│   │  PostgreSQL        │  Redis           │  Pinecone       │  S3       │   │
│   │  (Primary DB)      │  (Cache/Queue)   │  (Vectors)      │  (Files)  │   │
│   │                    │                  │                 │           │   │
│   │  • Orders          │  • Session       │  • Knowledge    │  • Images │   │
│   │  • Objects         │  • Rate limit    │  • Embeddings   │  • Documents │   │
│   │  • Technicians     │  • Real-time     │                 │  • Reports │   │
│   │  • ML History      │                  │                 │           │   │
│   └──────────────────────────────────────────────────────────────────────┘   │
│                                                                              │
│   ┌──────────────────────────────────────────────────────────────────────┐   │
│   │                    BACKGROUND JOBS (node-cron)                       │   │
│   │                                                                      │   │
│   │  • Retrain ML models (weekly)                                        │   │
│   │  • Generate maintenance predictions (daily)                          │   │
│   │  • Sync external data (hourly)                                       │   │
│   │  • Clean old data (daily)                                            │   │
│   └──────────────────────────────────────────────────────────────────────┘   │
│                                                                              │
└──────────────────────────────────────────────────────────────────────────────┘
```

5. AI-Komponenter (Detaljerat)

AI Planner - Input/Output Specifikation

Input:

```
interface PlanningInput {
  orders: Order[];              // Alla oplanerade ordrar
  technicians: Technician[];    // Tillgängliga tekniker med kompetens
  historicalData: HistoricalData; // Historiska slutförandetider
  realtimeData: {
    traffic: TrafficMatrix;     // Restidsmatris med trafikprognos
    weather: WeatherForecast;   // 7-dagars väderprognos
  };
  constraints: {
    timeWindows: TimeWindow[];  // Kundpreferenser
    maxHoursPerDay: number;     // Max arbetstid
    priorities: Priority[];      // Akut, normal, låg
    dependencies: Dependency[]; // Orderberoenden
    executionCodes: CodeMapping[]; // Kompetenskrav
    timeRestrictions: Restriction[]; // Parkeringsförbud, etc.
  };
}
```

Processing:

```
1. Feature Engineering
   - Extrahera features per order (typ, komplexitet, historik)
   - Extrahera features per tekniker (hastighet, specialisering)
   - Kombinera med externa features (väder, trafik)

2. Tidsestimering
   - Prediktera tid per uppgift med RandomForest
   - Beräkna konfidensintervall

3. Optimering (OR-Tools)
   - Bygg Vehicle Routing Problem (VRP)
   - Lägg till constraints (tidsfönster, kapacitet, kompetens)
   - Lös med guided local search

4. Ruttgenerering
   - Generera detaljerade rutter via Google Maps
   - Beräkna ETA per stopp

5. Förklaringar
   - Generera human-readable förklaringar för beslut
   - Identifiera trade-offs och alternativ
```

Output:

```typescript
interface PlanningOutput {
  schedule: ScheduleEntry[];    // Tilldelning per tekniker/dag/timme
  routes: Route[];              // Rutter med waypoints
  metrics: {
    efficiencyGain: number;     // % förbättring vs manuell
    distanceReduction: number;  // % minskning körsträcka
    workloadBalance: number;    // % balans mellan tekniker
    timeWindowCompliance: number; // % inom tidsfönster
  };
  explanations: string[];       // Förklaringar till beslut
  alternatives: PlanningOutput[]; // Alternativa planer
  confidence: number;           // Konfidens (0-1)
  planningTime: number;         // Tid i millisekunder
}
```

6. Datamodell (Utökad)

```sql
-- AI Planning History
CREATE TABLE ai_planning_history (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID REFERENCES tenants(id),
    week_start DATE NOT NULL,
    technicians JSONB NOT NULL,
    orders_count INTEGER NOT NULL,
    planning_time_ms INTEGER,
    metrics JSONB,
    explanations JSONB,
    applied BOOLEAN DEFAULT false,
    applied_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_planning_history_tenant ON ai_planning_history(tenant_id, week_start)
;

-- Task History for ML
CREATE TABLE task_history (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID REFERENCES tenants(id),
    task_id UUID REFERENCES work_orders(id),
    technician_id UUID REFERENCES users(id),
    object_type VARCHAR(50),
    object_count INTEGER,
    estimated_time INTEGER,
    actual_time INTEGER,
    weather JSONB,
    time_of_day VARCHAR(20),
    day_of_week INTEGER,
    completed_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_task_history_technician ON task_history(technician_id);
CREATE INDEX idx_task_history_object_type ON task_history(object_type);
CREATE INDEX idx_task_history_completed ON task_history(completed_at);

-- Predictive Maintenance
CREATE TABLE predictive_maintenance (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID REFERENCES tenants(id),
    object_id UUID REFERENCES objects(id),
    component VARCHAR(100),
    failure_probability_90d FLOAT,
    recommendation VARCHAR(200),
    urgency VARCHAR(20),
    proactive_cost INTEGER,
    reactive_cost INTEGER,
    potential_savings INTEGER,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_predictive_object ON predictive_maintenance(object_id);
CREATE INDEX idx_predictive_urgency ON predictive_maintenance(urgency);

-- Customer Communications
CREATE TABLE customer_communications (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID REFERENCES tenants(id),
```

```sql
    order_id UUID REFERENCES work_orders(id),
    customer_id UUID REFERENCES customers(id),
    customer_phone VARCHAR(20),
    channel VARCHAR(10), -- 'sms', 'email'
    direction VARCHAR(10), -- 'outbound', 'inbound'
    message_type VARCHAR(50),
    message TEXT,
    expects_response BOOLEAN DEFAULT false,
    response_received TEXT,
    sent_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_communications_order ON customer_communications(order_id);
CREATE INDEX idx_communications_customer ON customer_communications(customer_id);

-- Knowledge Base
CREATE TABLE knowledge_base (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID REFERENCES tenants(id),
    title VARCHAR(255),
    content TEXT,
    category VARCHAR(100),
    tags VARCHAR(255)[],
    source VARCHAR(100),
    author_id UUID REFERENCES users(id),
    views INTEGER DEFAULT 0,
    helpful_votes INTEGER DEFAULT 0,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_knowledge_tenant ON knowledge_base(tenant_id);
CREATE INDEX idx_knowledge_category ON knowledge_base(category);

-- ML Model Training Logs
CREATE TABLE model_training_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    model_name VARCHAR(100),
    version VARCHAR(50),
    training_samples INTEGER,
    metrics JSONB,
    hyperparameters JSONB,
    trained_at TIMESTAMP DEFAULT NOW()
);

-- Conversation History
CREATE TABLE conversation_history (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID REFERENCES tenants(id),
    conversation_id UUID,
    user_id UUID REFERENCES users(id),
    role VARCHAR(20), -- 'user', 'assistant'
    message TEXT,
    actions JSONB,
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_conversation_id ON conversation_history(conversation_id);
```

7. API-Endpoints

AI-Planering

```
POST /api/ai/plan-week:
  body:
    weekStart: "2026-02-24"
    technicianIds: ["uuid1", "uuid2"]
    constraints:
      maxHoursPerDay: 8
      overbookingTolerance: 1.1
      prioritizeUrgent: true
  response:
    schedule: [...]
    routes: [...]
    metrics: {...}
    explanations: [...]
    confidence: 0.95

POST /api/ai/plan-preview:
  description: "Snabb förhandsvisning utan att spara"

POST /api/ai/plan-apply:
  description: "Tillämpa AI-genererat schema"
  body:
    planningId: "uuid"
```

Tidsestimering

```
POST /api/ai/estimate-time:
  body:
    technicianId: "uuid"
    objectType: "molok"
    objectCount: 1
    weather: "sunny"
    timeOfDay: "morning"
  response:
    estimatedTime: 25
    confidenceInterval: [22, 28]
    similarTasksCount: 47
    explanation: "Baserat på 47 liknande uppgifter"

POST /api/ai/estimate-time/batch:
  body:
    tasks: [...]
  response:
    estimates: [...]
```

Konversationell Planering

```
POST /api/ai/conversation:
  body:
    conversationId: "uuid"
    message: "Planera nästa vecka för Johan"
    context:
      currentView: "week"
      weekNumber: 9
  response:
    response: "Jag hittar 47 ordrar..."
    intent: "plan_week"
    actions: [...]
    clarificationNeeded: false
```

Prediktivt Underhåll

```
GET /api/ai/predictive-maintenance:
  query:
    months: 3
    urgency: "high"
  response:
    objects: [
      {
        objectId: "uuid",
        component: "gasdämpare",
        failureProbability90d: 0.85,
        recommendation: "Byt nu",
        savings: 700
      }
    ]
    totalPotentialSavings: 6800

POST /api/ai/predictive-maintenance/create-orders:
  body:
    objectIds: ["uuid1", "uuid2"]
  response:
    createdOrders: 2
```

Kunskapsbas

```
POST /api/ai/knowledge/search:
  body:
    query: "Hur byter jag gasdämpare på molok?"
  response:
    answer: "Öppna locket helt..."
    sources: [
      { title: "Molok Manual", snippet: "..." }
    ]

POST /api/ai/knowledge/ingest:
  body:
    document:
      title: "Ny instruktion"
      content: "..."
      category: "manualer"
```

8. Implementationsordning (Fas 1)

Vecka 1-2: Setup och Infrastruktur

- [ ] **Dag 1-2: AI-infrastruktur**
- Skapa `server/services/ai/` katalogstruktur
- Konfigurera OpenAI, OR-Tools, scikit-learn
- Setup Pinecone vektor-databas
- [ ] **Dag 3-5: Database-utökning**
- Skapa nya tabeller (task_history, predictive_maintenance, etc.)
- Migrations och seeding
- Index-optimering
- [ ] **Dag 6-10: API-skelett**
- Skapa alla nya endpoints
- Autentisering och validering
- Error handling

Vecka 3-4: Prediktiv Tidsestimering

- [ ] **Dag 11-14: ML-modell**
- Implementera TimeEstimator-klass
- Feature engineering pipeline
- Träning med historisk data
- [ ] **Dag 15-18: API och integration**
- Implementera endpoints
- Integrera i WeekPlanner
- Cache-strategi
- [ ] **Dag 19-20: Validering**
- Cross-validation
- A/B-test mot standardtider
- Performance-optimering

Vecka 5-6: AI-Driven Planering (MVP)

- [ ] **Dag 21-24: Optimeringsmotor**
- OR-Tools integration
- VRP-modell
- Constraint-hantering
- [ ] **Dag 25-28: Frontend**
- AIPlanningDialog-komponent
- Kartvyn med rutter
- Preview och apply-flöde

- [ ] **Dag 29-30: Test**
- End-to-end tester
- Jämförelse med manuell planering

Vecka 7-8: Konversationell Planering

- [ ] **Dag 31-34: NLP-service**
- GPT-4 integration
- Intent-klassificering
- Tool-calling
- [ ] **Dag 35-38: Chat-komponent**
- ConversationalPlannerChat
- Realtidsuppdateringar
- Åtgärdsexekvering
- [ ] **Dag 39-40: Test**
- 50+ testfraser
- Konversationsflöden

Vecka 9-10: Automatisk Kundkommunikation

- [ ] **Dag 41-44: Event-system**
- Event-driven arkitektur
- Twilio-integration
- Resend-integration
- [ ] **Dag 45-48: Tvåvägskommunikation**
- Inkommande SMS-hantering
- AI-tolkning av svar
- Automatiska åtgärder
- [ ] **Dag 49-50: Test**
- End-to-end SMS-flöden
- Stresstest

Vecka 11-12: Integration och Polish

- [ ] **Dag 51-54: Integration**
- Integration med Fält App
- Synkronisering
- Real-time updates
- [ ] **Dag 55-58: Test och optimering**
- Lasttest
- Säkerhetsaudit
- Performance-optimering

- [ ] **Dag 59-60: Beta**
- Deploy till staging
- Beta-test med 3 kunder
- Feedback-insamling

---

9. Acceptanskriterier (Fas 1 - Komplett)

AI-Driven Planering

#	Kriterie	Mål	Test
1	Planeringstid	< 60 sekunder	Automatisk mätning
2	Effektivitetsvinst	> 20% vs manuell	A/B-test
3	Körsträckeminskning	> 25%	Jämför historik
4	Teknikerbalans	85-115%	Beräkning
5	Constraint-respekt	100%	Validering
6	Förklaringar	>= 3 per plan	Kontroll

Prediktiv Tidsestimering

#	Kriterie	Mål	Test
1	MAE	< 5 minuter	Cross-validation
2	Förbättring vs standard	> 25%	A/B-test
3	API-svarstid	< 500ms	Lasttest
4	Batch (100 tasks)	< 5 sekunder	Lasttest

Konversationell Planering

#	Kriterie	Mål	Test
1	Intent-förståelse	> 90%	50 testfraser
2	Svarstid	< 3 sekunder	Mätning
3	Åtgärder	100% korrekta	Validering

Automatisk Kundkommunikation

#	Kriterie	Mål	Test
1	Leveranstid SMS	< 30 sekunder	Mätning
2	Korrekt ETA	±5 minuter	Jämför ankomst
3	AI-tolkning av svar	> 90%	Validering
4	Minskning kund-samtal	> 50%	Jämför historik

---

10. Performance-Krav

Funktion	Mål	Max
AI-planering (100 ordrar)	30s	60s
AI-planering (500 ordrar)	120s	180s
Tidsestimering (single)	200ms	500ms
Tidsestimering (batch 100)	2s	5s
Konversation	2s	4s
Knowledge search	1s	3s
Dashboard-laddning	2s	5s
API (enkel)	100ms	300ms

---

11. Skalbarhet

Dimension	Nuvarande	Mål (12 mån)
Ordrar/vecka	500	5,000
Tekniker	20	200
Objekt	5,000	50,000
Samtidiga användare	20	200
API-anrop/sekund	10	100
Tenants	5	50

---

12. Monitoring och Logging

Metrics att spåra

```
// server/services/monitoring/metrics.ts

const AI_METRICS = {
  // Planering
  'ai.planning.duration_ms': Histogram,
  'ai.planning.efficiency_gain_pct': Gauge,
  'ai.planning.orders_planned': Counter,

  // Tidsestimering
  'ai.estimation.mae_minutes': Gauge,
  'ai.estimation.requests': Counter,

  // Konversation
  'ai.conversation.messages': Counter,
  'ai.conversation.intent_accuracy': Gauge,

  // Prediktivt underhåll
  'ai.maintenance.predictions': Counter,
  'ai.maintenance.accuracy': Gauge,

  // Kostnader
  'ai.openai.tokens_used': Counter,
  'ai.openai.cost_usd': Counter,
};
```

Logging-strategi

```
// Strukturerad loggning
logger.info('AI Planning completed', {
  tenantId,
  ordersPlanned: result.schedule.length,
  planningTimeMs: result.planningTime,
  efficiencyGain: result.metrics.efficiencyGain,
  confidence: result.confidence,
});

// Error tracking
logger.error('AI Planning failed', {
  tenantId,
  error: error.message,
  stack: error.stack,
  input: sanitizedInput,
});
```

---

13. Continuous Learning

Automatisk omträning

```
// server/jobs/retrain-models.ts

// Veckovis omträning av tidsestimator
cron.schedule('0 2 * * 0', async () => {
  const tenants = await getTenants();

  for (const tenant of tenants) {
    const data = await getTrainingData(tenant.id);
    const metrics = await timeEstimator.train(data);

    await logTrainingRun('time_estimator', tenant.id, metrics);

    if (metrics.mae < previousMetrics.mae) {
      await deployModel('time_estimator', tenant.id);
    }
  }
});

// Daglig uppdatering av prediktivt underhåll
cron.schedule('0 3 * * *', async () => {
  const tenants = await getTenants();

  for (const tenant of tenants) {
    const predictions = await maintenanceService.predictAll(tenant.id);
    await savePredictions(tenant.id, predictions);
  }
});
```

Feedback-loop

```
// Samla feedback från planerare
router.post('/api/ai/planning/feedback', async (req, res) => {
  const { planningId, wasUseful, adjustmentsMade, comments } = req.body;

  await db.insert('ai_planning_feedback', {
    planningId,
    wasUseful,
    adjustmentsMade,
    comments,
    userId: req.userId,
  });

  res.json({ success: true });
});
```

---

14. Säkerhet

Data-isolation

- Multi-tenant isolation på alla AI-queries
- Tenant-filter i Pinecone-queries
- Separata ML-modeller per tenant (option)

API-säkerhet

```
// Rate limiting för AI-endpoints
const aiRateLimiter = rateLimit({
  windowMs: 60 * 1000, // 1 minut
  max: 30, // max 30 AI-anrop per minut
  keyGenerator: (req) => req.tenantId,
});

router.use('/api/ai/*', aiRateLimiter);
```

GDPR

- Kunddata anonymiseras i ML-träning
- Konversationshistorik raderas efter 30 dagar
- Röstdata transkriberas och raderas direkt

---

15. Dokumentation

API-dokumentation

- [ ] OpenAPI/Swagger spec för alla AI-endpoints
- [ ] Exempel på requests/responses
- [ ] Rate limits och kvoter

Utvecklardokumentation

- [ ] Arkitekturbeskrivning
- [ ] ML-modell-dokumentation
- [ ] Deployment-guide

Användardokumentation

- [ ] Admin-guide för AI-funktioner
- [ ] Konversationsexempel
- [ ] Troubleshooting

---

16. Framtida Utveckling (Fas 2+)

Fas 2 (Månad 4-6)

- Prediktivt underhåll
- Smart kunskapsbas
- Förbättrad NLP

Fas 3 (Månad 7-9)

- Intelligent lagerhantering
- Immersiv kundportal
- Advanced analytics

Fas 4 (Månad 10-12)

- Intelligent overbooking
- Full autonomi
- White-label AI

Sammanfattning för Replit-utvecklare

Huvudfokus Fas 1:
1. **AI-Driven Planering** - OR-Tools + ML för optimal schemaläggning
2. **Prediktiv Tidsestimering** - RandomForest för noggranna tidsuppskattningar
3. **Konversationell Planering** - GPT-4 för naturligt språk
4. **Automatisk Kundkommunikation** - Event-driven SMS/Email

Tech stack:
- Express.js + TypeScript backend
- React + shadcn/ui frontend
- PostgreSQL + Redis + Pinecone
- OpenAI API (GPT-4, Embeddings)
- OR-Tools (optimization)
- scikit-learn / Prophet (ML)
- Twilio + Resend (kommunikation)

Kritiska framgångsfaktorer:
- Planeringstid < 60 sekunder

- Effektivitetsvinst > 20%
- Tidsestimering MAE < 5 minuter
- Konversationsförståelse > 90%

Dokumentversion: 1.0
Skapad: 2026-02-21
Författare: DeepAgent